

TERRAFORM CHALLENGE

Write a code using Terraform that deploys and achieves the following:-

1. Create an EC2 Instance on AWS Cloud using the below-mentioned ubuntu AMI and instance_type. Or, (You can use of your choice as well) `ami = "ami-0461b1436c9b3c1a3"`
`instance_type = "t3.micro"`

Terraform process to create one Ubuntu EC2 instance on AWS using:

- **AMI:** ami-0461b1436c9b3c1a3
- **Instance type:** t3.micro
- **Region:** us-east-1
- **Terraform provider:** AWS
- **OS:** Ubuntu
- **Access:** SSH
- **Security Group:** Allow SSH (22) and HTTP (80)

Important: AMI IDs are region-specific. The AMI ami-0461b1436c9b3c1a3 must exist in us-east-1. If it doesn't, use an Ubuntu AMI available in your selected region.

Step 1: Create a Terraform project

Since you're working from Windows/Git Bash, create a folder:

```
cd /c/Terraform
```

```
mkdir ec2-ubuntu-terraform
```

```
cd ec2-ubuntu-terraform
```

Check:

```
pwd
```

You should see:

```
/c/Terraform/ec2-ubuntu-terraform
```

Step 2: Create the Terraform files

Create these files:

ec2-ubuntu-terraform/

|

├── provider.tf

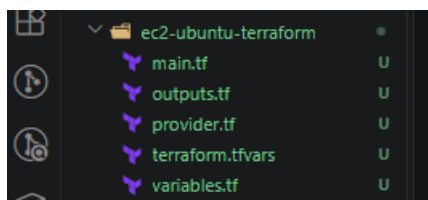
├── variables.tf

├── main.tf

├── outputs.tf

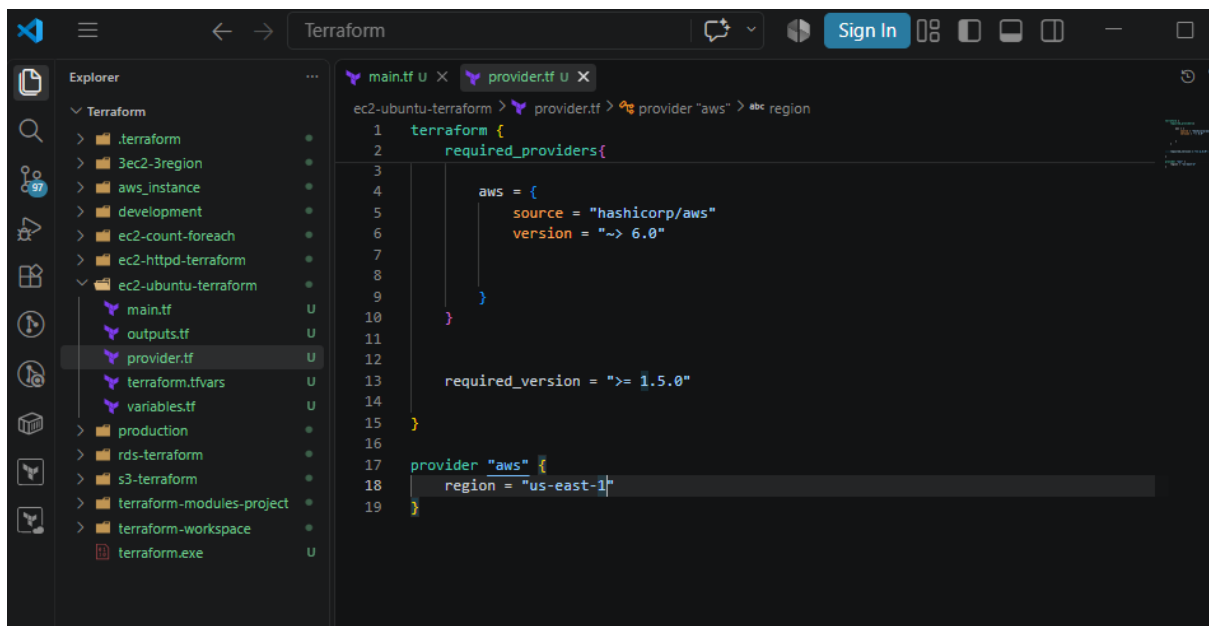
└── terraform.tfvars

You can create them in VS Code.



Step 3: Create provider.tf

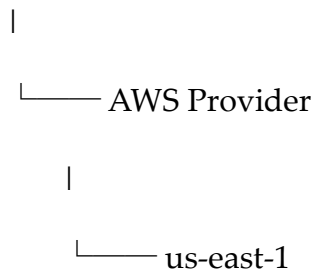
Put this code inside provider.tf



Explanation

This tells Terraform:

Terraform



Step 4: Create variables.tf

Put this code inside variables.tf

```
1 variable "ami" {
2     description = "Ubuntu Ami ID"
3     type = string
4     default = "ami-0b6d9d3d33ba97d99"
5 }
6
7 variable "instance_type" {
8     description = "EC2 instance type"
9     type = string
10    default = "t3.micro"
11 }
12
13 variable "instance_name" {
14     description = "name of EC2 instance"
15     type = string
16     default = "Terraform-Ubuntu-Server"
17 }
18
19 }
```

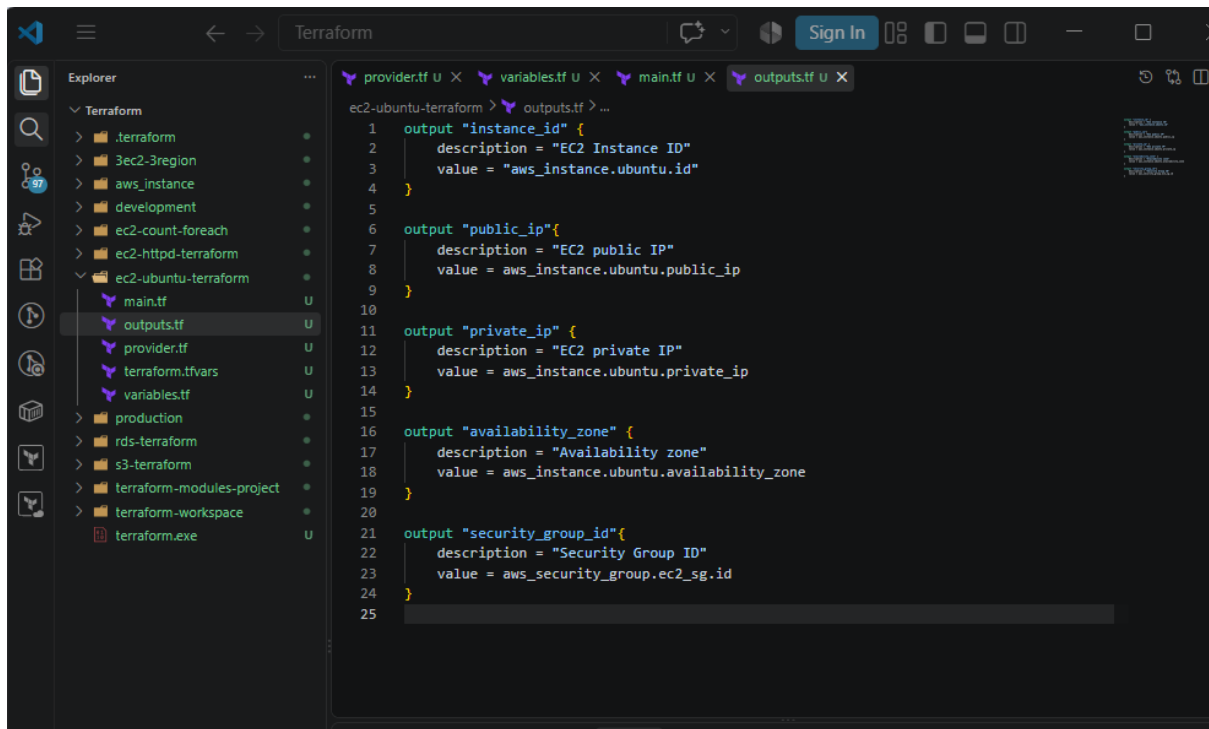
Step 5: Create main.tf

Now create the actual EC2 infrastructure.

```
1 # Get the default VPC
2 data "aws_vpc" "default" {
3     default = true
4 }
5
6 # Get the default subnet
7 data "aws_subnets" "default" {
8     filter {
9         name = "vpc-id"
10        values = [data.aws_vpc.default.id]
11    }
12 }
13
14 # Security Group
15 resource "aws_security_group" "ec2_sg" {
16     name = "terraform-ubuntu-sg"
17     description = "Allow SSH and HTTP"
18     vpc_id = data.aws_vpc.default.id
19 }
20
21 # Allow HTTP
22 ingress {
23     description = "HTTP"
24     from_port = 80
25     to_port = 80
26     protocol = "tcp"
27     cidr_blocks = ["0.0.0.0/0"]
28 }
29
30 # Allow all outbound traffic
31 egress {
32     from_port = 0
33     to_port = 0
34     protocol = "-1"
35     cidr_blocks = ["0.0.0.0/0"]
36 }
37
38 tags = {
39     Name = "Terraform-Ubuntu-SG"
40 }
41
42 # EC2 Instance
43 resource "aws_instance" "ubuntu" {
44     ami = var.ami
45     instance_type = var.instance_type
46     subnet_id = data.aws_subnets.default.ids[0]
47 }
```

Step 6: Create outputs.tf

Put this code inside outputs.tf



Step 7: Create terraform.tfvars

Put:

ami = "ami-0461b1436c9b3c1a3"

instance_type = "t2.micro"

instance_name = "Terraform-Ubuntu-Server"

Your final project should look like:

ec2-ubuntu-terraform

|

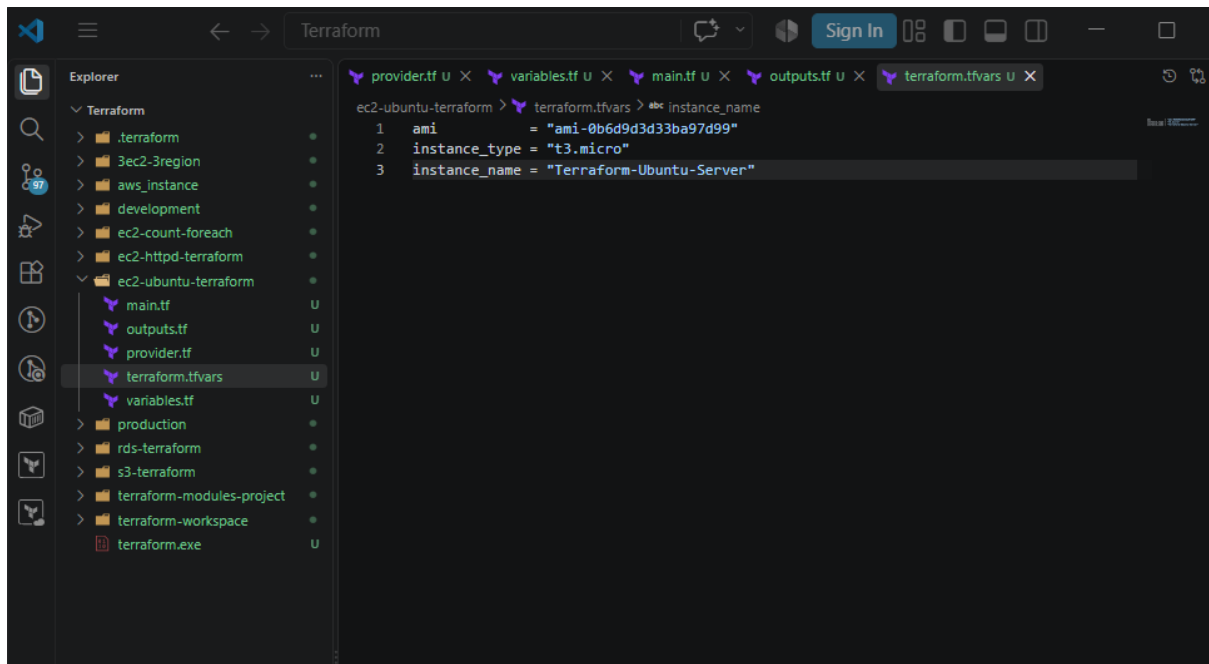
├── provider.tf

├── variables.tf

├── main.tf

├── outputs.tf

└── terraform.tfvars



Step 8: Configure AWS credentials

Before running Terraform, make sure your AWS CLI is configured.

Check:

```
aws --version
```

Then:

```
aws configure
```

It will ask:

AWS Access Key ID:

AWS Secret Access Key:

Default region name:

Default output format:

Enter:

AWS Access Key ID: YOUR_ACCESS_KEY

AWS Secret Access Key: YOUR_SECRET_KEY

Default region name: us-east-1

Default output format: json

Better option for your AWS lab

If your EC2/Terraform environment already has an **IAM role**, prefer the role instead of storing access keys.

Step 9: Verify AWS authentication

Run:

```
aws sts get-caller-identity
```

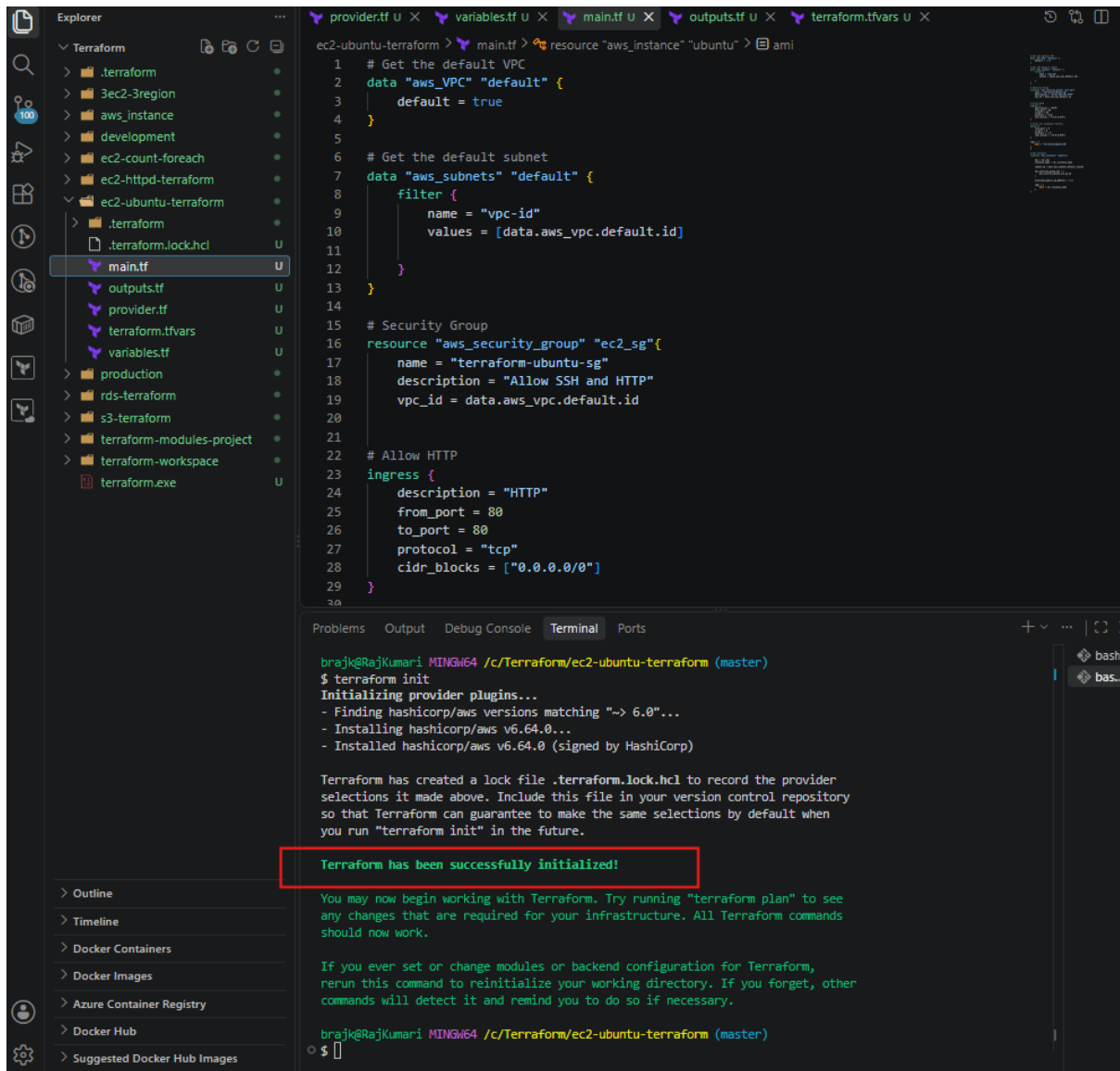
Step 10: Run terraform init

Inside:

```
/c/Terraform/ec2-ubuntu-terraform
```

run:

```
terraform init
```

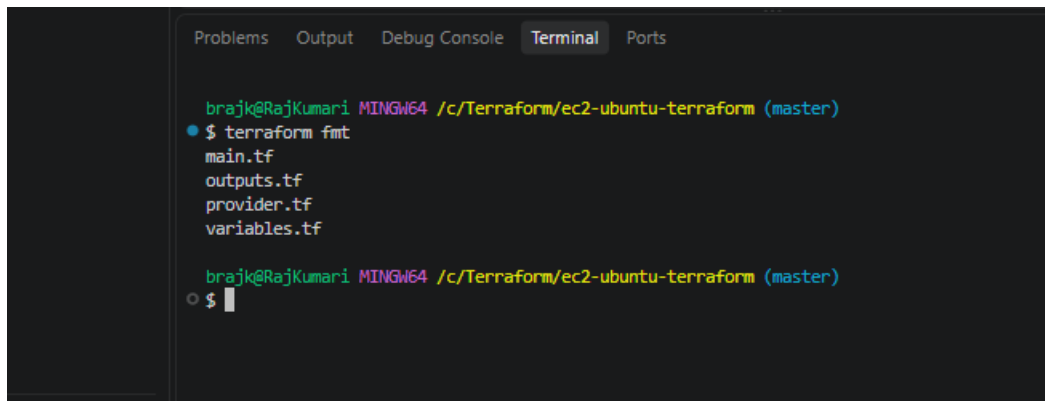


Step 11: Run Terraform format

Run:

```
terraform fmt
```

This automatically formats your .tf files.

A terminal window with a dark background. The prompt is 'brajk@RajKumari MINGW64 /c/Terraform/ec2-ubuntu-terraform (master)'. The command '\$ terraform fmt' has been executed, and the output lists the files that were formatted: 'main.tf', 'outputs.tf', 'provider.tf', and 'variables.tf'.

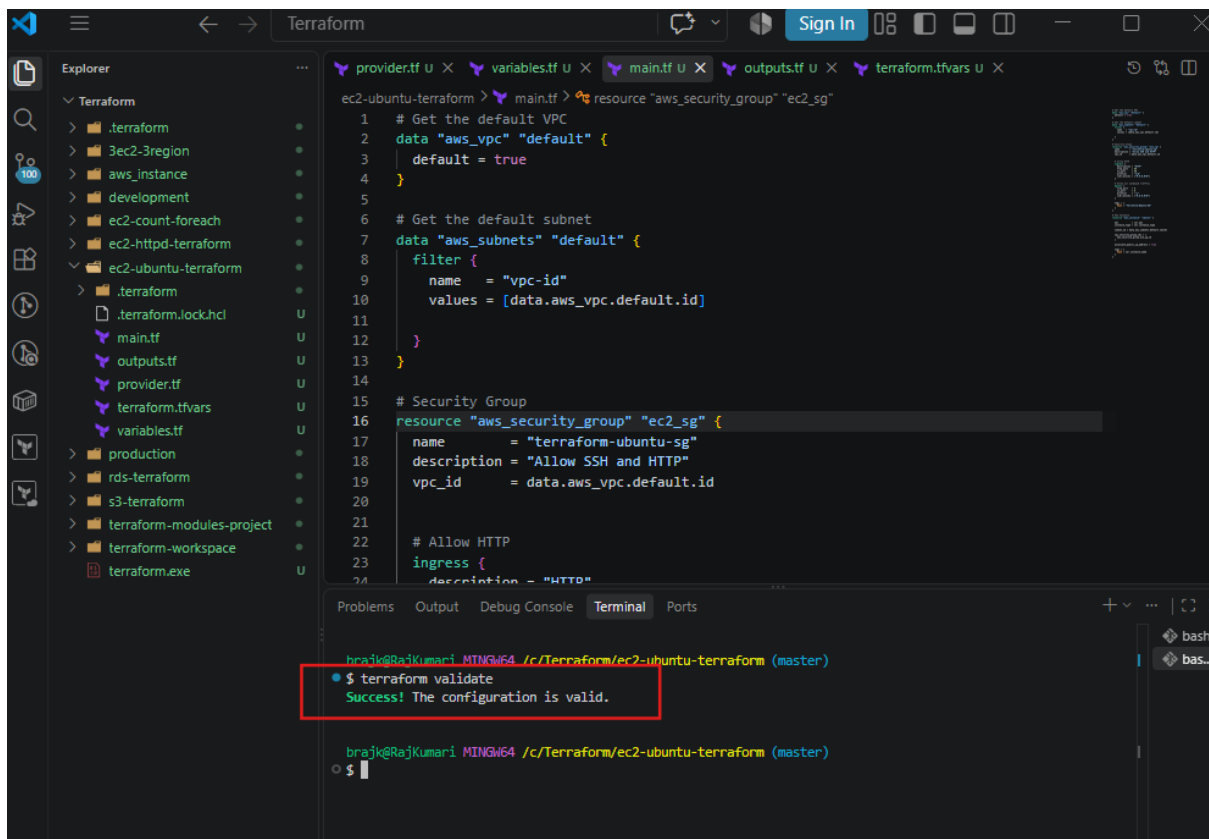
```
brajk@RajKumari MINGW64 /c/Terraform/ec2-ubuntu-terraform (master)
$ terraform fmt
main.tf
outputs.tf
provider.tf
variables.tf

brajk@RajKumari MINGW64 /c/Terraform/ec2-ubuntu-terraform (master)
$
```

Step 12: Validate the Terraform code

Run:

terraform validate

A screenshot of the Visual Studio Code editor. The Explorer pane on the left shows a project structure for 'Terraform' with various subfolders and files. The main editor area shows the 'main.tf' file with Terraform code for an AWS VPC and a Security Group. The Output pane at the bottom shows the command '\$ terraform validate' and the message 'Success! The configuration is valid.' highlighted with a red box.

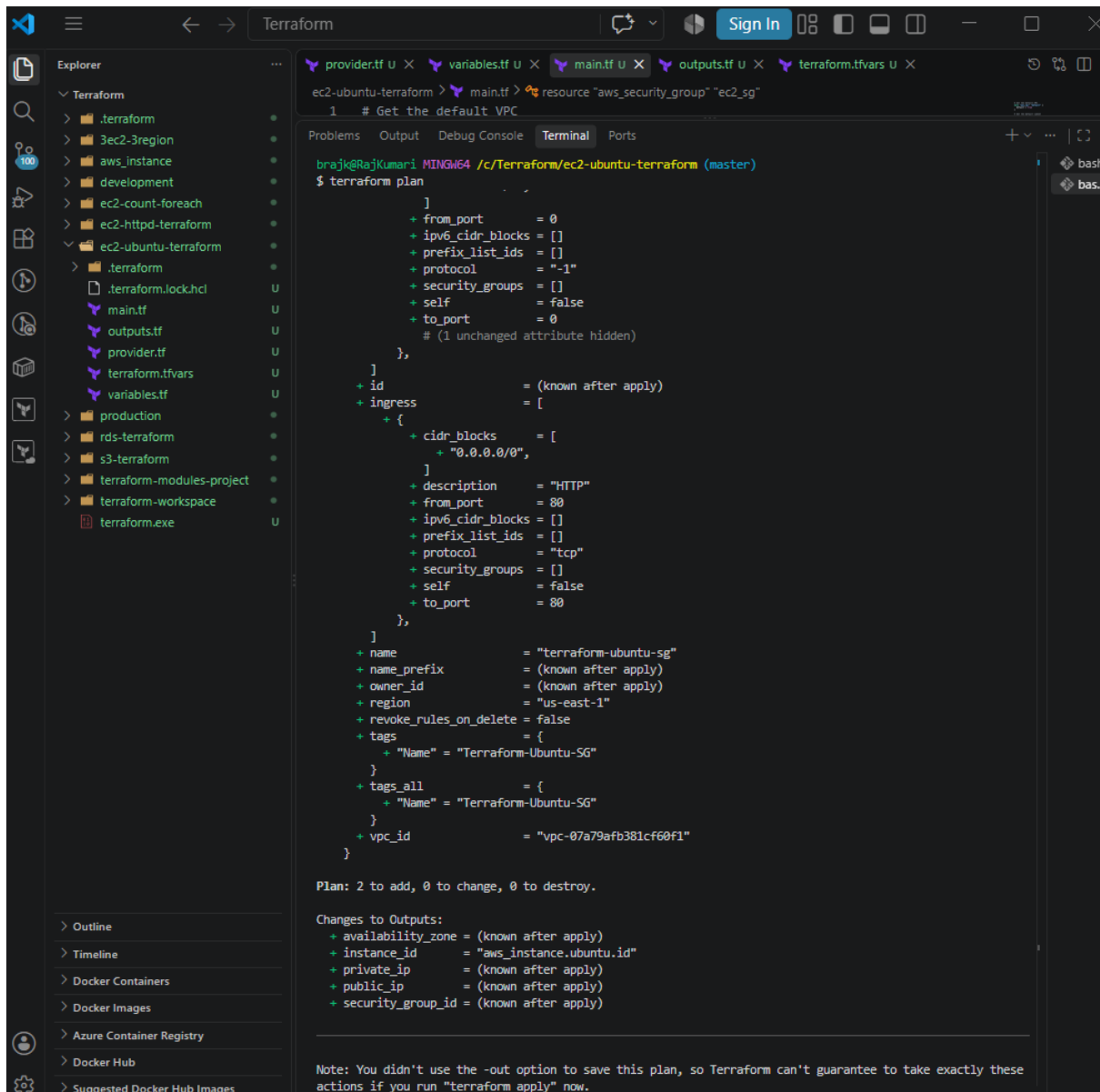
```
brajk@RajKumari MINGW64 /c/Terraform/ec2-ubuntu-terraform (master)
$ terraform validate
Success! The configuration is valid.

brajk@RajKumari MINGW64 /c/Terraform/ec2-ubuntu-terraform (master)
$
```

Step 13: Run terraform plan

Now run:

terraform plan



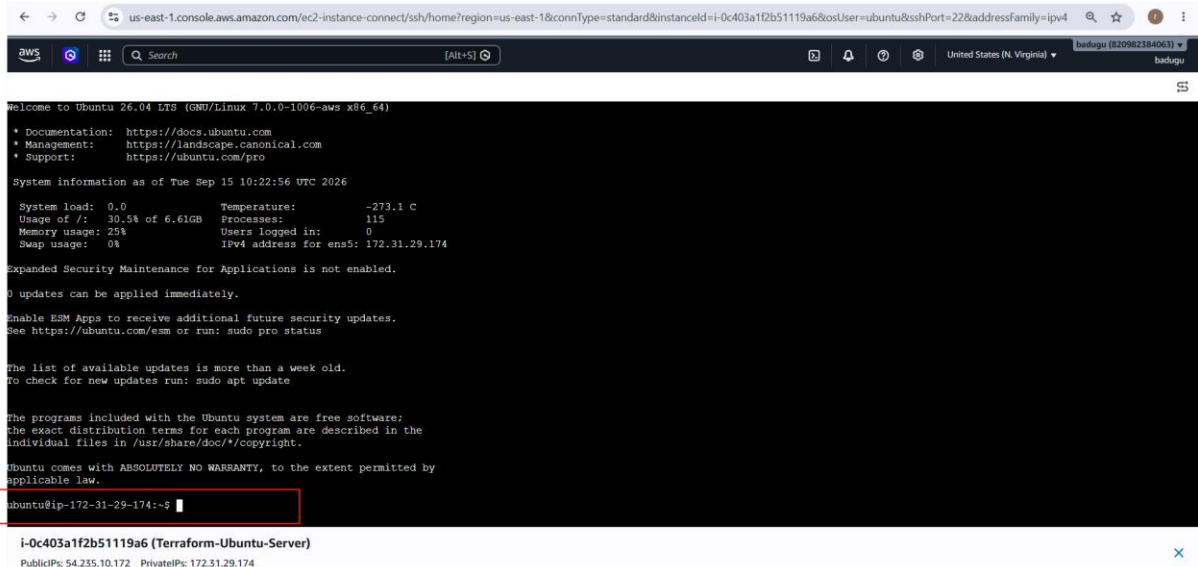
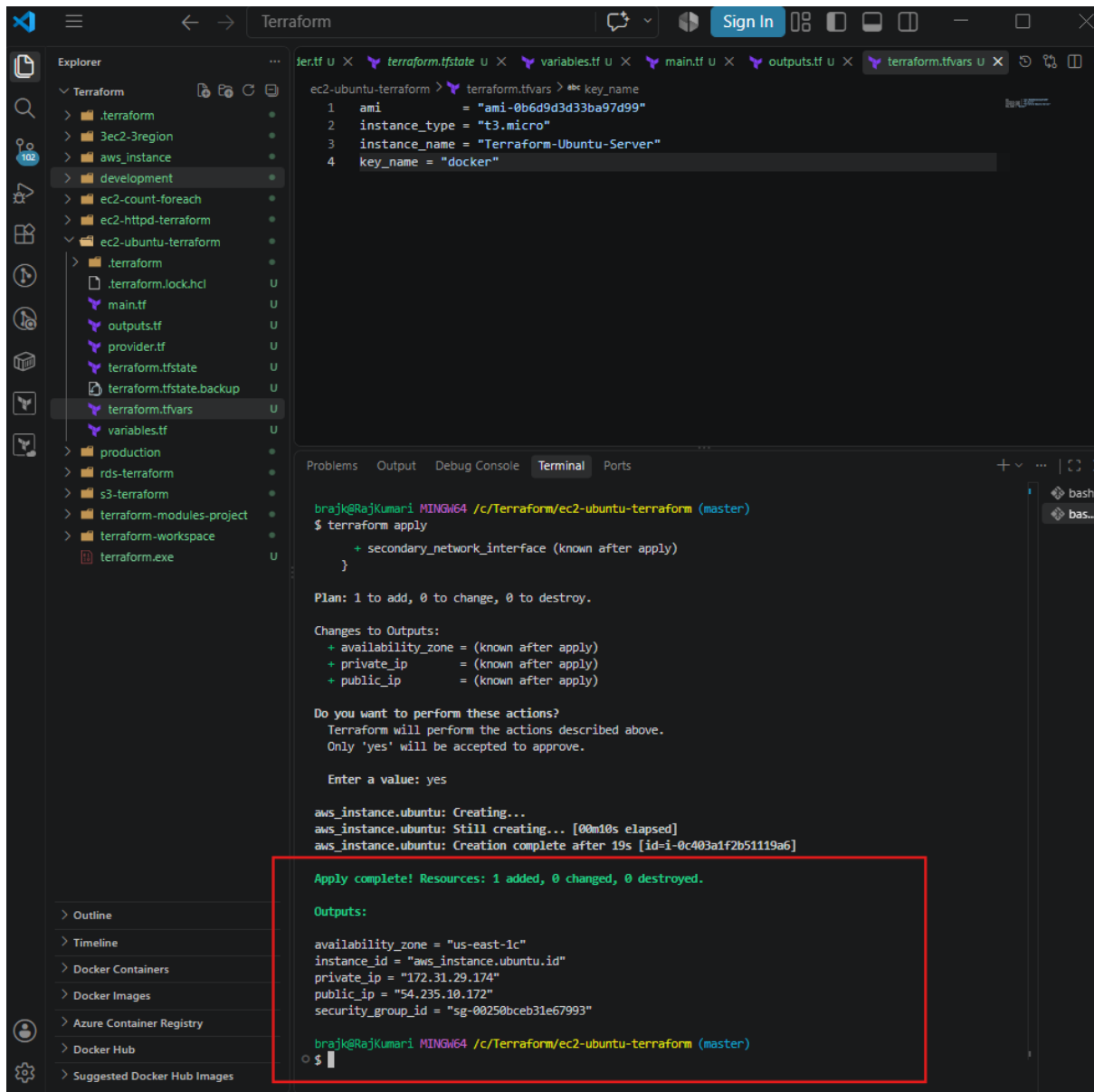
Step 14: Apply the infrastructure

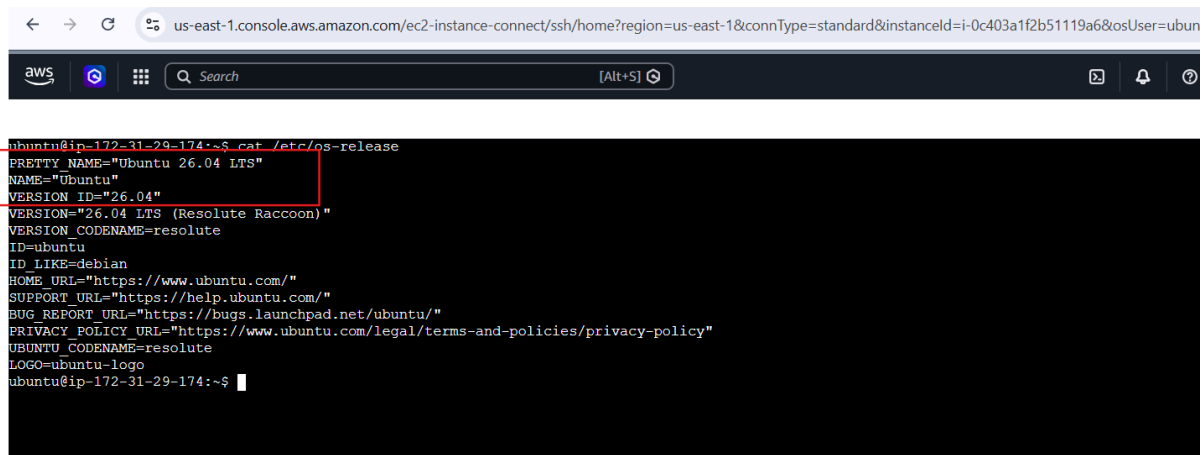
Run:

```
terraform apply
```

Step 15: Check the output

At the end, you should see something similar to:





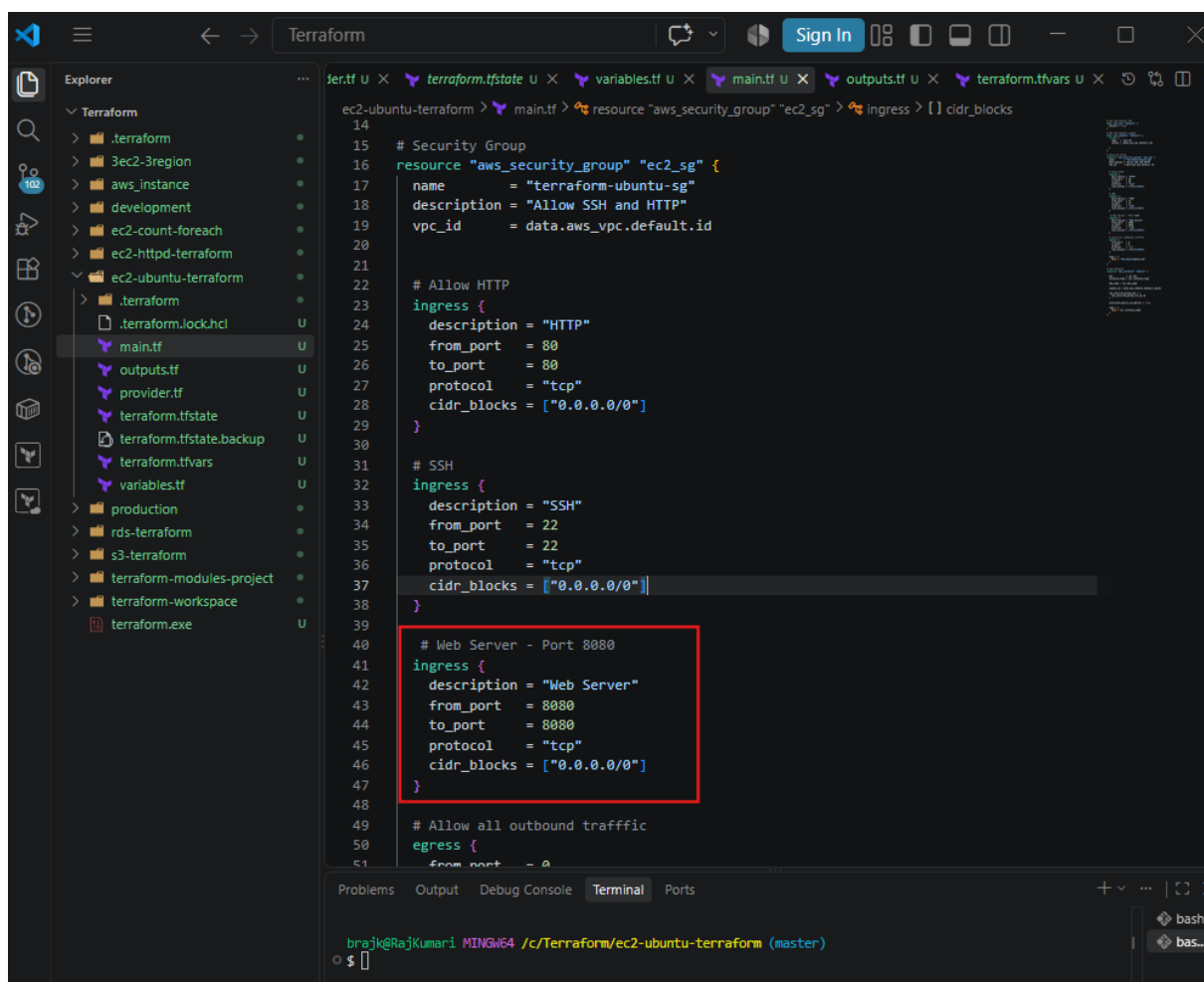
```
ubuntu@ip-172-31-29-174:~$ cat /etc/os-release
PRETTY_NAME="Ubuntu 26.04 LTS"
NAME="Ubuntu"
VERSION_ID="26.04"
VERSION="26.04 LTS (Resolute Raccoon)"
VERSION_CODENAME=resolute
ID=ubuntu
ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
UBUNTU_CODENAME=resolute
LOGO=ubuntu-logo
ubuntu@ip-172-31-29-174:~$
```

2. Install a web server on the same EC2 Instance. The web server should listen on port 8080.

To install **Apache HTTP Server** and configure it to listen on **port 8080**.

Step 1: Add port 8080 to your Terraform Security Group

In your main.tf, update the security group:



```
ec2-ubuntu-terraform > main.tf > resource "aws_security_group" "ec2_sg" > ingress > [ ] cidr_blocks
14
15 # Security Group
16 resource "aws_security_group" "ec2_sg" {
17   name           = "terraform-ubuntu-sg"
18   description    = "Allow SSH and HTTP"
19   vpc_id         = data.aws_vpc.default.id
20
21   # Allow HTTP
22   ingress {
23     description = "HTTP"
24     from_port   = 80
25     to_port     = 80
26     protocol    = "tcp"
27     cidr_blocks = ["0.0.0.0/0"]
28   }
29
30   # SSH
31   ingress {
32     description = "SSH"
33     from_port   = 22
34     to_port     = 22
35     protocol    = "tcp"
36     cidr_blocks = ["0.0.0.0/0"]
37   }
38
39   # Web Server - Port 8080
40   ingress {
41     description = "Web Server"
42     from_port   = 8080
43     to_port     = 8080
44     protocol    = "tcp"
45     cidr_blocks = ["0.0.0.0/0"]
46   }
47
48   # Allow all outbound traffic
49   egress {
50     from_port   = 0
51     to_port     = 0
```

Step 2: Apply the Security Group change

From your Terraform directory:

terraform fmt

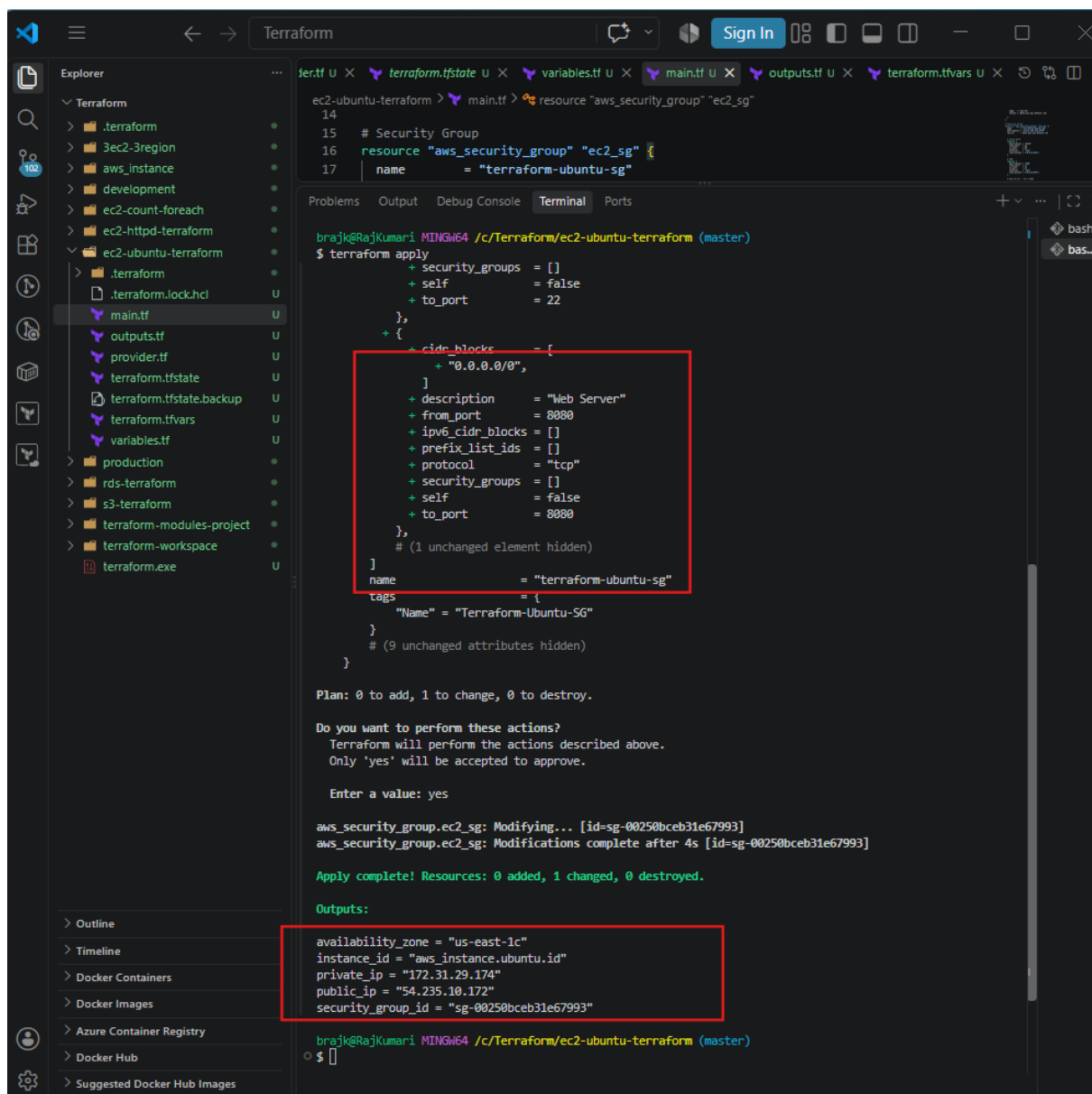
terraform validate

terraform plan

terraform apply

Enter:

yes



The screenshot shows a VS Code editor with a Terraform project. The Explorer pane on the left shows the file structure, including .terraform, 3ec2-3region, aws_instance, development, ec2-count-foreach, ec2-httpd-terraform, ec2-ubuntu-terraform, .terraform.lock.hcl, main.tf, outputs.tf, provider.tf, terraform.tfstate, terraform.tfstate.backup, terraform.tfvars, variables.tf, production, rds-terraform, s3-terraform, terraform-modules-project, terraform-workspace, and terraform.exe. The main editor shows the main.tf file with the following content:

```
ec2-ubuntu-terraform > main.tf > resource "aws_security_group" "ec2_sg"
14
15 # Security Group
16 resource "aws_security_group" "ec2_sg" {
17   name = "terraform-ubuntu-sg"
```

The terminal pane at the bottom shows the output of the terraform apply command. The output is as follows:

```
brajk@RajKumari MINGW64 /c/Terraform/ec2-ubuntu-terraform (master)
$ terraform apply
+ security_groups = []
+ self           = false
+ to_port        = 22
+ },
+ {
+   cidr_blocks = [
+     "0.0.0.0/0",
+   ]
+   description = "Web Server"
+   from_port   = 8080
+   ipv6_cidr_blocks = []
+   prefix_list_ids = []
+   protocol      = "tcp"
+   security_groups = []
+   self          = false
+   to_port       = 8080
+ },
# (1 unchanged element hidden)
]
name = "terraform-ubuntu-sg"
tags = {
  "Name" = "Terraform-Ubuntu-SG"
}
# (9 unchanged attributes hidden)
}

Plan: 0 to add, 1 to change, 0 to destroy.

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

aws_security_group.ec2_sg: Modifying... [id=sg-00250bceb31e67993]
aws_security_group.ec2_sg: Modifications complete after 4s [id=sg-00250bceb31e67993]

Apply complete! Resources: 0 added, 1 changed, 0 destroyed.

Outputs:
availability_zone = "us-east-1c"
instance_id      = "aws_instance.ubuntu.id"
private_ip       = "172.31.29.174"
public_ip        = "54.235.10.172"
security_group_id = "sg-00250bceb31e67993"

brajk@RajKumari MINGW64 /c/Terraform/ec2-ubuntu-terraform (master)
$
```

Step 3: SSH into the EC2 instance

Once port 22 is working, connect to your Ubuntu EC2.

For example:

```
ssh -i docker.pem ubuntu@54.235.10.172
```

Important: Your Terraform instance was configured with:

```
key_name = "docker"
```

So you need the private key corresponding to the AWS key pair ubuntu-key. docker.pem will only work if it belongs to the same EC2 key pair.

Step 4: Update Ubuntu packages

Inside the EC2:

```
sudo apt update
```

```
aws | [icons] | United States (N. Virginia) | badugu (820982384063) | badugu
```

```
Get:32 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute-backports/universe amd64 Components [1056 B]
Get:33 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute-backports/universe amd64 c-n-f Metadata [116 B]
Get:34 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute-backports/restricted amd64 Components [216 B]
Get:35 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute-backports/restricted amd64 c-n-f Metadata [120 B]
Get:36 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute-backports/multiverse amd64 Components [216 B]
90% [4 Packages store 0 B] [36 Components-amd64 216 B/216 B 100%] [8 Packages 92.4 kB/474 kB 20%]
Get:37 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute-backports/multiverse amd64 c-n-f Metadata [120 B]
Get:38 http://security.ubuntu.com/ubuntu resolute-security/main Translation-en [111 kB]
Get:39 http://security.ubuntu.com/ubuntu resolute-security/main amd64 Components [46.6 kB]
Get:40 http://security.ubuntu.com/ubuntu resolute-security/main amd64 c-n-f Metadata [4616 B]
Get:41 http://security.ubuntu.com/ubuntu resolute-security/universe amd64v3 Packages [173 kB]
Get:42 http://security.ubuntu.com/ubuntu resolute-security/universe Translation-en [54.4 kB]
Get:43 http://security.ubuntu.com/ubuntu resolute-security/universe amd64 Components [43.4 kB]
Get:44 http://security.ubuntu.com/ubuntu resolute-security/universe amd64 c-n-f Metadata [3552 B]
Get:45 http://security.ubuntu.com/ubuntu resolute-security/restricted amd64v3 Packages [399 kB]
Get:46 http://security.ubuntu.com/ubuntu resolute-security/restricted Translation-en [77.7 kB]
Get:47 http://security.ubuntu.com/ubuntu resolute-security/multiverse amd64v3 Packages [10.9 kB]
Get:48 http://security.ubuntu.com/ubuntu resolute-security/multiverse Translation-en [2844 B]
Get:49 http://security.ubuntu.com/ubuntu resolute-security/multiverse amd64 Components [212 B]
Get:50 http://security.ubuntu.com/ubuntu resolute-security/multiverse amd64 c-n-f Metadata [120 B]
Fetched 31.7 MB in 4s (7834 kB/s)
169 packages can be upgraded. Run 'apt list --upgradable' to see them.
ubuntu@ip-172-31-29-174:~$
```

```
i-0c403a1f2b51119a6 (Terraform-Ubuntu-Server)
PublicIPs: 54.235.10.172 PrivateIPs: 172.31.29.174
```

Step 5: Install Apache

Run:

```
sudo apt install apache2 -y
```

Check Apache:

```
sudo systemctl status apache2
```

You should see:

Active: active (running)

```
us-east-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?regi...
aws
United States (N. Virginia)
badugu (820982384063)
badugu

abuntu@ip-172-31-29-174:~$ sudo systemctl status apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
   Active: active (running) since Tue 2026-09-15 10:46:32 UTC; 1min 13s ago
 Invocation: i60d5eadb69e41f5b72e7c47324dede5
    Docs: https://httpd.apache.org/docs/2.4/
   Main PID: 2461 (apache2)
  Status: "Total requests: 0; Idle/Busy workers 100/0;Requests/sec: 0; Bytes served/sec: 0 B/sec"
     Tasks: 55 (limit: 627)
    Memory: 5.8M (peak: 6M)
       CPU: 97ms
    CGroup: /system.slice/apache2.service
            └─2461 /usr/sbin/apache2 -k start -DFOREGROUND
              └─2477 /usr/sbin/apache2 -k start -DFOREGROUND
                └─2478 /usr/sbin/apache2 -k start -DFOREGROUND

Sep 15 10:46:32 ip-172-31-29-174 systemd[1]: Starting apache2.service - The Apache HTTP Server.
..
Sep 15 10:46:32 ip-172-31-29-174 systemd[1]: Started apache2.service - The Apache HTTP Server.
abuntu@ip-172-31-29-174:~$
```

i-0c403a1f2b51119a6 (Terraform-Ubuntu-Server)

PublicIPs: 54.235.10.172 PrivateIPs: 172.31.29.174

Step 6: Configure Apache to listen on port 8080

By default Apache listens on port 80.



```
aws | [icons] | United States (N. Virginia) | badugu (820982384063) | badugu

# If you just change the port or add more ports here, you will likely also
# have to change the VirtualHost statement in
# /etc/apache2/sites-enabled/000-default.conf

Listen 8080

<IfModule ssl_module>
    Listen 443
</IfModule>

<IfModule mod_gnutls.c>
    Listen 443
</IfModule>
```

Step 7: Change the Apache VirtualHost

Now edit:

Find:

<VirtualHost *:80>

Change it to:

<VirtualHost *:8080>

Save and exit.

```
aws | [icons] | United States (N. Virginia) | badugu (820982384063) | badugu

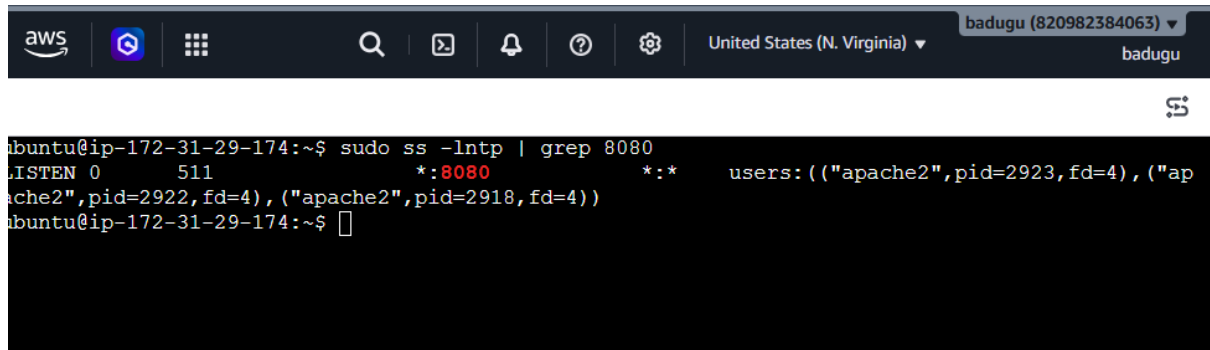
ubuntu@ip-172-31-29-174:~$ sudo systemctl restart apache2
ubuntu@ip-172-31-29-174:~$ sudo systemctl status apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
   Active: active (running) since Tue 2026-09-15 10:53:59 UTC; 6s ago
     Invocation: 7f92c2efb91b49d18b160b332022e270
       Docs: https://httpd.apache.org/docs/2.4/
    Main PID: 2918 (apache2)
      Status: "Processing requests..."
        Tasks: 55 (limit: 627)
       Memory: 5.4M (peak: 5.8M)
          CPU: 59ms
       CGroup: /system.slice/apache2.service
               └─2918 /usr/sbin/apache2 -k start -DFOREGROUND
                 └─2922 /usr/sbin/apache2 -k start -DFOREGROUND
                   └─2923 /usr/sbin/apache2 -k start -DFOREGROUND

Sep 15 10:53:59 ip-172-31-29-174 systemd[1]: Starting apache2.service - The Apache HTTP Server.
...
Sep 15 10:53:59 ip-172-31-29-174 systemd[1]: Started apache2.service - The Apache HTTP Server.
ubuntu@ip-172-31-29-174:~$
```

Step 9: Verify Apache is listening on 8080

Run:

```
sudo ss -lntp | grep 8080
```



```
aws | [icons] | United States (N. Virginia) | badugu (820982384063) | badugu

ubuntu@ip-172-31-29-174:~$ sudo ss -lntp | grep 8080
LISTEN 0      511          *:8080      *:      users: (("apache2",pid=2923,fd=4), ("ap
ache2",pid=2922,fd=4), ("apache2",pid=2918,fd=4))
ubuntu@ip-172-31-29-174:~$
```

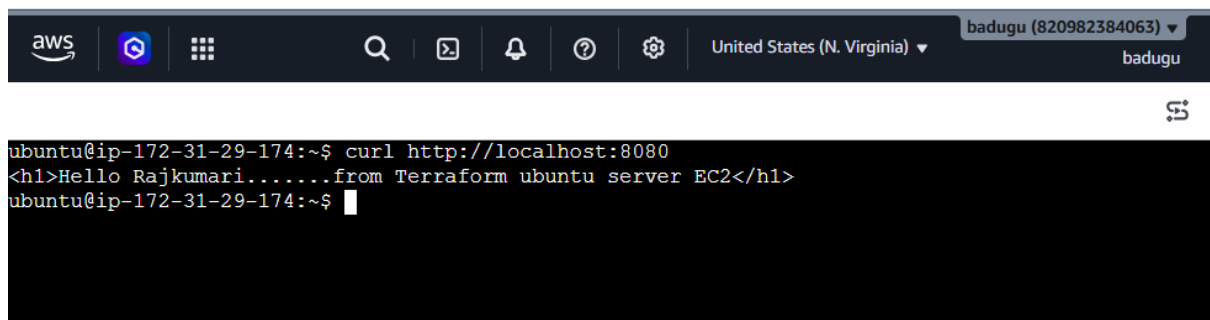
Step 10: Create a test web page

Run: `echo "<h1>Hello from Terraform EC2</h1>" | sudo tee /var/www/html/index.html`

Step 11: Test from the EC2 itself

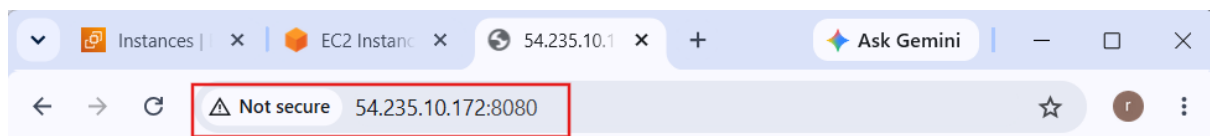
Run:

```
curl http://localhost:8080
```



```
aws | [icons] | United States (N. Virginia) | badugu (820982384063) | badugu

ubuntu@ip-172-31-29-174:~$ curl http://localhost:8080
<h1>Hello Rajkumari.....from Terraform ubuntu server EC2</h1>
ubuntu@ip-172-31-29-174:~$
```



Hello Rajkumari.....from Terraform ubuntu server EC2

3. Create a Docker-compose file for the project: <https://github.com/betawins/cricbuzz-docker-hello-world.git> and point it to port 8000.

It is a **Go Hello World web server** and its README shows that the application listens on **container port 8080**; the repository's Docker example maps host port 8000 to container port 8080.

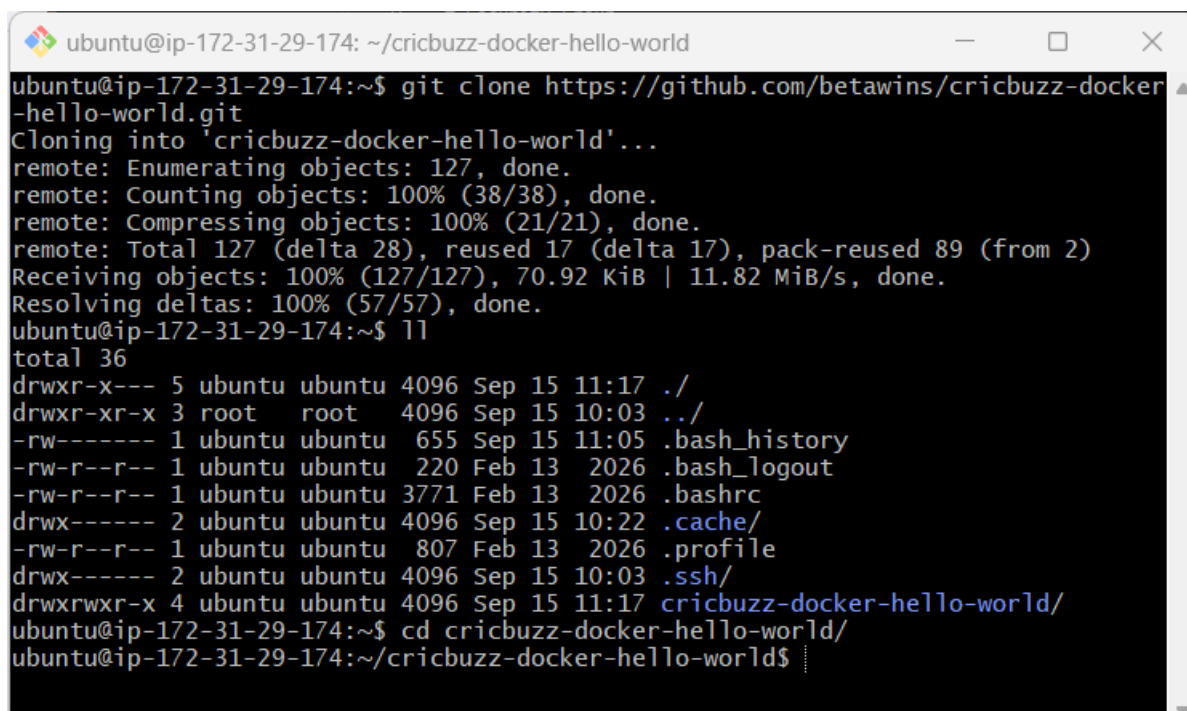
Docker Compose uses the ports mapping in the form **host-port:container-port**.

1. Clone the project

On your EC2 instance or local machine:

```
git clone https://github.com/betawins/cricbuzz-docker-hello-world.git
```

```
cd cricbuzz-docker-hello-world
```

A terminal window titled 'ubuntu@ip-172-31-29-174: ~/cricbuzz-docker-hello-world' showing the execution of 'git clone' and 'ls'. The output of 'git clone' shows the repository being cloned into 'cricbuzz-docker-hello-world'. The output of 'ls' shows the contents of the directory, including a 'cricbuzz-docker-hello-world/' subdirectory.

```
ubuntu@ip-172-31-29-174: ~/$ git clone https://github.com/betawins/cricbuzz-docker-hello-world.git
Cloning into 'cricbuzz-docker-hello-world'...
remote: Enumerating objects: 127, done.
remote: Counting objects: 100% (38/38), done.
remote: Compressing objects: 100% (21/21), done.
remote: Total 127 (delta 28), reused 17 (delta 17), pack-reused 89 (from 2)
Receiving objects: 100% (127/127), 70.92 KiB | 11.82 MiB/s, done.
Resolving deltas: 100% (57/57), done.
ubuntu@ip-172-31-29-174: ~/$ ll
total 36
drwxr-x--- 5 ubuntu ubuntu 4096 Sep 15 11:17 ./
drwxr-xr-x 3 root root 4096 Sep 15 10:03 ../
-rw----- 1 ubuntu ubuntu 655 Sep 15 11:05 .bash_history
-rw-r--r-- 1 ubuntu ubuntu 220 Feb 13 2026 .bash_logout
-rw-r--r-- 1 ubuntu ubuntu 3771 Feb 13 2026 .bashrc
drwx----- 2 ubuntu ubuntu 4096 Sep 15 10:22 .cache/
-rw-r--r-- 1 ubuntu ubuntu 807 Feb 13 2026 .profile
drwx----- 2 ubuntu ubuntu 4096 Sep 15 10:03 .ssh/
drwxrwxr-x 4 ubuntu ubuntu 4096 Sep 15 11:17 cricbuzz-docker-hello-world/
ubuntu@ip-172-31-29-174: ~/$ cd cricbuzz-docker-hello-world/
ubuntu@ip-172-31-29-174: ~/cricbuzz-docker-hello-world$
```

```
ubuntu@ip-172-31-29-174: ~/cricbuzz-docker-hello-world
ubuntu@ip-172-31-29-174:~/cricbuzz-docker-hello-world$ ll
total 120
drwxrwxr-x 4 ubuntu ubuntu 4096 Sep 15 11:17 ./
drwxr-x--- 5 ubuntu ubuntu 4096 Sep 15 11:17 ../
drwxrwxr-x 7 ubuntu ubuntu 4096 Sep 15 11:17 .git/
-rw-rw-r-- 1 ubuntu ubuntu   8 Sep 15 11:17 .gitattributes
drwxrwxr-x 3 ubuntu ubuntu 4096 Sep 15 11:17 .github/
-rw-rw-r-- 1 ubuntu ubuntu  259 Sep 15 11:17 .gitignore
-rw-rw-r-- 1 ubuntu ubuntu  177 Sep 15 11:17 Dockerfile
-rw-rw-r-- 1 ubuntu ubuntu 11357 Sep 15 11:17 LICENSE
-rw-rw-r-- 1 ubuntu ubuntu  142 Sep 15 11:17 Makefile
-rw-rw-r-- 1 ubuntu ubuntu  2329 Sep 15 11:17 README.md
-rw-rw-r-- 1 ubuntu ubuntu  2067 Sep 15 11:17 go.mod
-rw-rw-r-- 1 ubuntu ubuntu 60305 Sep 15 11:17 go.sum
-rw-rw-r-- 1 ubuntu ubuntu  1437 Sep 15 11:17 hello_world.go
-rw-rw-r-- 1 ubuntu ubuntu  1087 Sep 15 11:17 hello_world_test.go
ubuntu@ip-172-31-29-174:~/cricbuzz-docker-hello-world$ |
```

2. Create docker-compose.yml

Inside the project directory:

vi docker-compose.yml

```
ubuntu@ip-172-31-29-174: ~/cricbuzz-docker-hello-world
services:
  cricbuzz-app:
    build:
      context: .
      container_name: cricbuzz-hello-world
    port:
      - "8000:8080"
~
~
~
~
~
~
~
~
```

What does this mean?

ports:

- "8000:8080"

means:

EC2 Host

Docker Container

Port 8000 -----> Port 8080

|

|

Go application

So when you access:

http://<EC2-PUBLIC-IP>:8000

Docker forwards the request to:

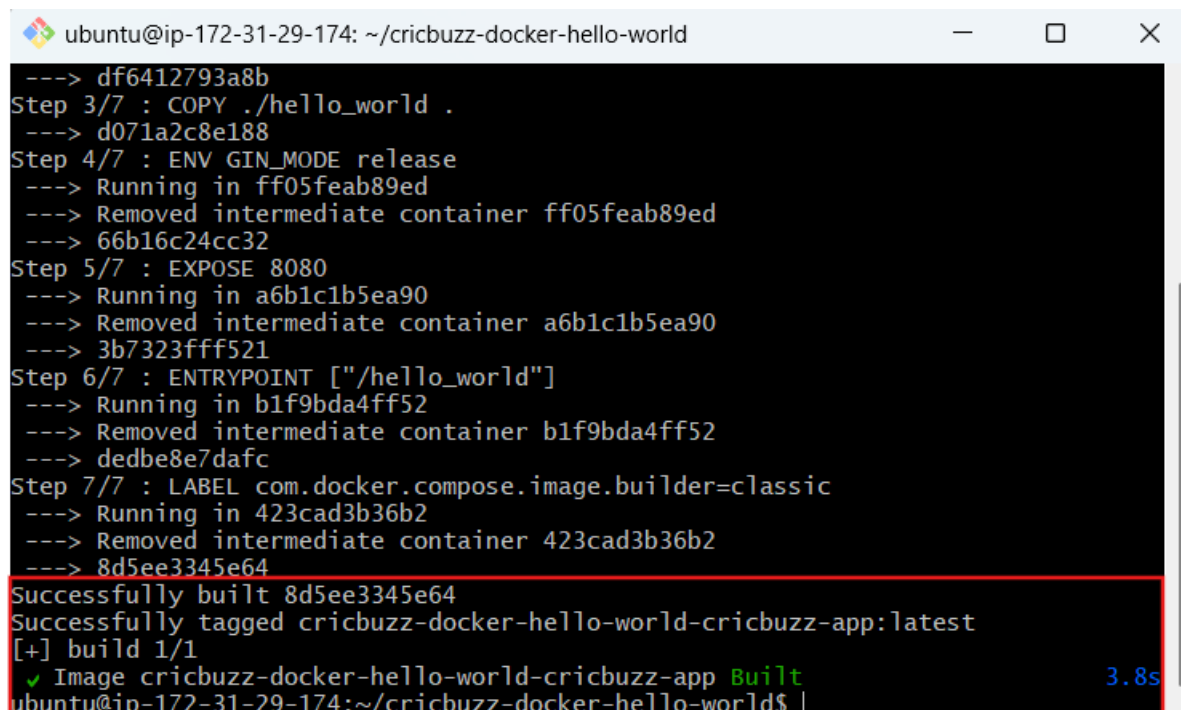
container:8080

The repository itself documents the equivalent Docker mapping 8000:8080.

Build the Docker image

Now run:

docker-compose build

A terminal window titled 'ubuntu@ip-172-31-29-174: ~/cricbuzz-docker-hello-world' showing the output of a Docker build command. The build proceeds through seven steps: copying files, setting environment variables, exposing port 8080, and setting the entrypoint. The final output shows the image was successfully built and tagged as 'cricbuzz-docker-hello-world-cricbuzz-app:latest' in 3.8 seconds.

```
ubuntu@ip-172-31-29-174: ~/cricbuzz-docker-hello-world
---> df6412793a8b
Step 3/7 : COPY ./hello_world .
---> d071a2c8e188
Step 4/7 : ENV GIN_MODE release
---> Running in ff05feab89ed
---> Removed intermediate container ff05feab89ed
---> 66b16c24cc32
Step 5/7 : EXPOSE 8080
---> Running in a6b1c1b5ea90
---> Removed intermediate container a6b1c1b5ea90
---> 3b7323fff521
Step 6/7 : ENTRYPOINT ["/hello_world"]
---> Running in b1f9bda4ff52
---> Removed intermediate container b1f9bda4ff52
---> dedbe8e7dafc
Step 7/7 : LABEL com.docker.compose.image.builder=classic
---> Running in 423cad3b36b2
---> Removed intermediate container 423cad3b36b2
---> 8d5ee3345e64
Successfully built 8d5ee3345e64
Successfully tagged cricbuzz-docker-hello-world-cricbuzz-app:latest
[+] build 1/1
✓ Image cricbuzz-docker-hello-world-cricbuzz-app Built 3.8s
ubuntu@ip-172-31-29-174:~/cricbuzz-docker-hello-world$ |
```

Step 5 — Start the container

If the Docker build succeeds:

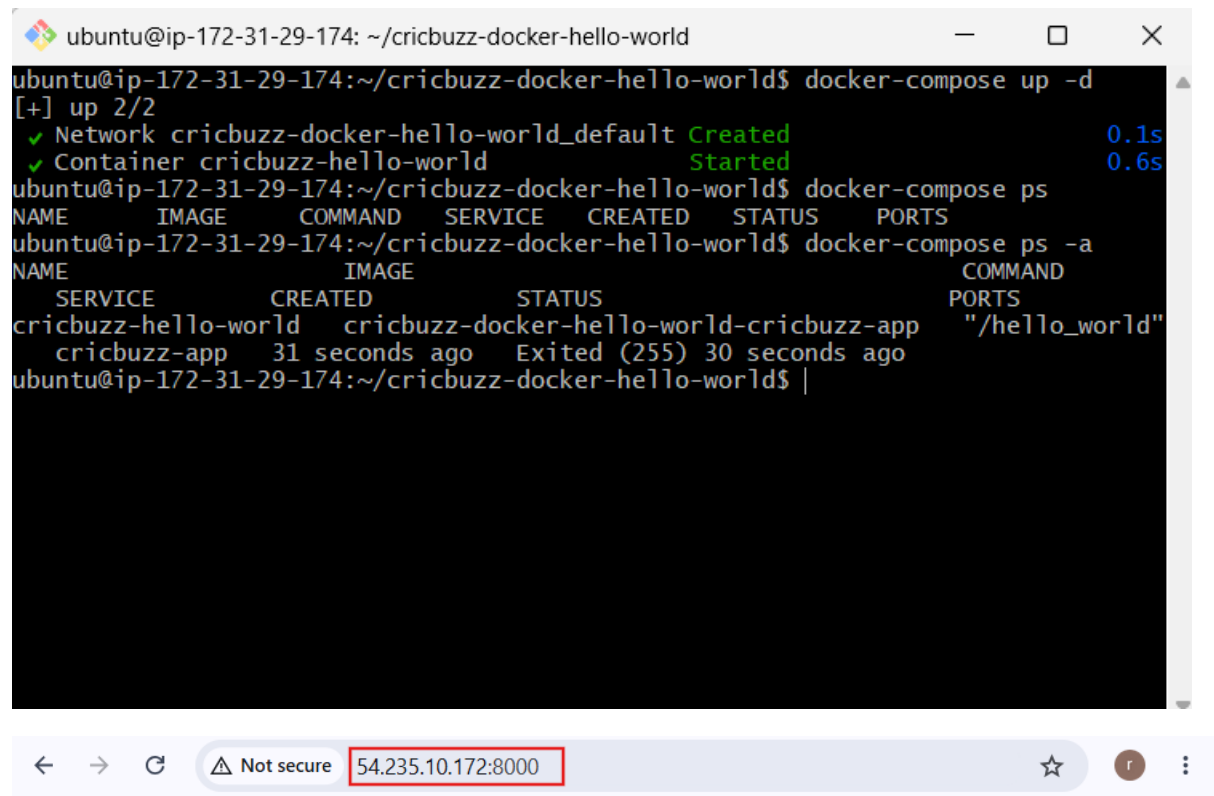
docker-compose up -d

Check:

docker-compose ps

5. Test from the EC2

Run:



The image shows a terminal window and a web browser. The terminal window displays the output of the following commands:

```
ubuntu@ip-172-31-29-174: ~/cricbuzz-docker-hello-world
ubuntu@ip-172-31-29-174:~/cricbuzz-docker-hello-world$ docker-compose up -d
[+] up 2/2
✓ Network cricbuzz-docker-hello-world_default Created 0.1s
✓ Container cricbuzz-hello-world Started 0.6s
ubuntu@ip-172-31-29-174:~/cricbuzz-docker-hello-world$ docker-compose ps
NAME          IMAGE          COMMAND          SERVICE  CREATED   STATUS    PORTS
ubuntu@ip-172-31-29-174:~/cricbuzz-docker-hello-world$ docker-compose ps -a
NAME          IMAGE          COMMAND          SERVICE  CREATED   STATUS    PORTS
cricbuzz-hello-world  cricbuzz-docker-hello-world-cricbuzz-app  "/hello_world"
cricbuzz-app      31 seconds ago  Exited (255) 30 seconds ago
ubuntu@ip-172-31-29-174:~/cricbuzz-docker-hello-world$ |
```

The web browser shows the URL `54.235.10.172:8000` and displays the text "Hello, World!"

Hello, World!

4. Make the web server point to the above service so that 8080 returns the response from the Docker service.

Now we will connect the **Apache web server on port 8080** to the Docker Compose service running on **port 8000**.

The final flow will be:

Browser

|

| http://EC2-PUBLIC-IP:8080

v

Apache :8080

|

| Reverse Proxy

v

Docker Compose :8000

|

v

Cricbuzz container :8080

|

v

"Hello, World!"

So when you open:

http://<EC2-PUBLIC-IP>:8080

Apache will return the response from the Docker service.

1. Make sure Docker Compose is running

Go to your project:

```
cd ~/cricbuzz-docker-hello-world
```

Check:

```
docker compose ps
```

You should see something similar to:

NAME	STATUS	PORTS
cricbuzz-hello-world	Up	0.0.0.0:8000->8080/tcp

Test it directly:

```
curl http://localhost:8000
```

Expected:

Hello, World!

If this doesn't work, fix Docker first before configuring Apache.

```
ubuntu@ip-172-31-29-174: ~/cricbuzz-docker-hello-world
ubuntu@ip-172-31-29-174:~/cricbuzz-docker-hello-world$ docker-compose ps -a
NAME                IMAGE                                     COMMAND
SERVICE            CREATED          STATUS          PORTS          COMMAND
cricbuzz-hello-world cricbuzz-docker-hello-world-cricbuzz-app "/hello_world"
cricbuzz-app        6 minutes ago   Up 5 minutes    0.0.0.0:8000->8080/tcp, [::]:8000->8080/tcp
ubuntu@ip-172-31-29-174:~/cricbuzz-docker-hello-world$ |
```

2. Install Apache

If Apache is not installed:

```
ubuntu@ip-172-31-29-174: ~/cricbuzz-docker-hello-world
ubuntu@ip-172-31-29-174:~/cricbuzz-docker-hello-world$ sudo apt update
Hit:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute InRelease
Hit:2 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute-updates InRelease
Hit:3 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute-backports InRelease
Hit:4 http://security.ubuntu.com/ubuntu resolute-security InRelease
161 packages can be upgraded. Run 'apt list --upgradable' to see them.
apache2 is already the newest version (2.4.66-2ubuntu2.4).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 161
ubuntu@ip-172-31-29-174:~/cricbuzz-docker-hello-world$ sudo apt install apache2 -y
Hit:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute InRelease
Hit:2 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute-updates InRelease
Hit:3 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute-backports InRelease
Hit:4 http://security.ubuntu.com/ubuntu resolute-security InRelease
161 packages can be upgraded. Run 'apt list --upgradable' to see them.
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 161
ubuntu@ip-172-31-29-174:~/cricbuzz-docker-hello-world$ sudo systemctl status apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset:
   Active: active (running) since Tue 2026-09-15 11:48:39 UTC; 19min ago
   Invocation: ea2b799ee518452394fe99214b93b7b2
     Docs: https://httpd.apache.org/docs/2.4/
   Main PID: 8623 (apache2)
   Status: "Total requests: 2; Idle/Busy workers 100/0;Requests/sec: 0.00172;
   Tasks: 55 (limit: 627)
   Memory: 6.9M (peak: 7.4M)
   CPU: 168ms
   CGroup: /system.slice/apache2.service
```

3. Enable Apache reverse-proxy modules

We need Apache to forward requests to the Docker service.

Run:

```
ubuntu@ip-172-31-29-174: ~/cricbuzz-docker-hello-world
<VirtualHost *:80>
    ServerName 54.235.10.172

    ProxyPreserveHost on

    ProxyPass / http://127.0.0.1:8000/
    ProxyPassReverse / http://127.0.0.1:8000/

    ErrorLog ${APACHE_LOG_DIR}/cricbuzz-error.log
    CustomLog ${APACHE_LOG_DIR}/cricbuzz-access.log combined
</VirtualHost>
~
~
~
~
~
```

```
ubuntu@ip-172-31-28-253: ~
ubuntu@ip-172-31-28-253:~$ sudo a2enmod proxy
Enabling module proxy.
To activate the new configuration, you need to run:
    systemctl restart apache2
ubuntu@ip-172-31-28-253:~$ sudo systemctl restart apache2
ubuntu@ip-172-31-28-253:~$ sudo a2enmod proxy
Module proxy already enabled
ubuntu@ip-172-31-28-253:~$ sudo a2enmod proxy_http
Considering dependency proxy for proxy_http:
Module proxy already enabled
Enabling module proxy_http.
To activate the new configuration, you need to run:
    systemctl restart apache2
ubuntu@ip-172-31-28-253:~$ sudo apache2ctl -M | grep proxy
proxy_module (shared)
proxy_http_module (shared)
ubuntu@ip-172-31-28-253:~$ |
```

4. Configure Apache for port 8080

Open:

`sudo nano /etc/apache2/ports.conf`

Make sure this line exists:

Listen 8080

```
ubuntu@ip-172-31-28-253: ~
GNU nano 8.7.1 /etc/apache2/ports.conf *
# If you just change the port or add more ports here, you will likely also
# have to change the VirtualHost statement in
# /etc/apache2/sites-enabled/000-default.conf

Listen 8080

<IfModule ssl_module>
    Listen 443
</IfModule>

<IfModule mod_gnutls.c>
    Listen 443
</IfModule>
```

5. Configure the reverse proxy

Open:

```
sudo nano /etc/apache2/sites-available/000-default.conf
```

6. Test Apache configuration

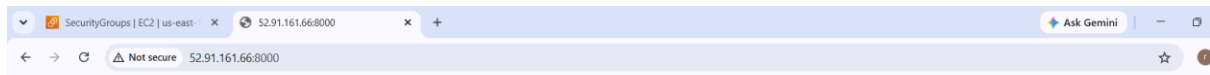
Before restarting:

```
ubuntu@ip-172-31-28-253: ~
ubuntu@ip-172-31-28-253:~$ sudo apache2ctl configtest
Syntax OK
ubuntu@ip-172-31-28-253:~$ sudo systemctl restart apache2
ubuntu@ip-172-31-28-253:~$ sudo systemctl status apache2 --no-pager
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: e
   Active: active (running) since Wed 2026-09-16 09:08:36 UTC; 5s ago
   Invocation: 214a67dd8a544898b11b81fb70d903b5
     Docs: https://httpd.apache.org/docs/2.4/
   Main PID: 35617 (apache2)
    Status: "Processing requests..."
     Tasks: 55 (limit: 627)
    Memory: 6M (peak: 6M)
       CPU: 62ms
    CGroup: /system.slice/apache2.service
            └─35617 /usr/sbin/apache2 -k start -DFOREGROUND
              └─35621 /usr/sbin/apache2 -k start -DFOREGROUND
                └─35622 /usr/sbin/apache2 -k start -DFOREGROUND

Sep 16 09:08:36 ip-172-31-28-253 systemd[1]: Starting apache2.service - The ....
Sep 16 09:08:36 ip-172-31-28-253 systemd[1]: Started apache2.service - The A...er.
Hint: Some lines were ellipsized, use -l to show in full.
ubuntu@ip-172-31-28-253:~$
```

7. Check Docker

Now go to your project:



Hello, World!

5. Configure to Save the Logs in the location `/var/log/cb.log`

Requirement is to **save the Cricbuzz application logs in `/var/log/cb.log`**, the clean approach is to configure Docker to write the container's stdout/stderr to that host file.

Your architecture will be:

Cricbuzz application



Docker container



Docker logs



`/var/log/cb.log`

Step 1 — Make sure the container is running

From your project directory:

```
cd ~/cricbuzz-docker-hello-world
```

Check:

```
docker compose ps
```

You should see:

```
cricbuzz-hello-world ... Up
```

If it is not running, fix that first.

```
✓ Container cricbuzz-hello-world Started 0.5s
ubuntu@ip-172-31-28-253:~/cricbuzz-docker-hello-world$ docker compose ps
NAME                IMAGE                                COMMAND                  S
ERVICE            CREATED          STATUS          PORTS
cricbuzz-hello-world  cricbuzz-docker-hello-world-cricbuzz  "/hello_world"        c
ricbuzz            7 seconds ago   Up 7 seconds    0.0.0.0:8000->8080/tcp, [::]:8000->8080
/tcp
ubuntu@ip-172-31-28-253:~/cricbuzz-docker-hello-world$
```

Step 2 — Create the log file

Run:

```
sudo touch /var/log/cb.log
```

Check:

```
ls -lh /var/log/cb.log
```

You should see something like:

```
-rw-r--r-- 1 root root 0 ... /var/log/cb.log
```

Step 3 — Configure Docker Compose logging

Open your Compose file:

```
vi docker-compose.yml
```

```
ubuntu@ip-172-31-28-253: ~/cricbuzz-docker-hello-world
services:
  cricbuzz:
    build:
      context:
        container_name: cricbuzz-hello-world

    ports:
      - "8000:8080"

    logging:
      driver: "json-file"
      options:

        max-size: "10m"
        max-file: "3"

    volumes:
      - /var/log/cb.log:/var/log/cb.log
```

Important point

The volumes line:

```
- /var/log/cb.log:/var/log/cb.log
```

mounts the EC2 host file:

```
/var/log/cb.log
```

inside the container at:

```
/var/log/cb.log
```

However, **mounting the file alone does not make the application write logs to it.**

Your Go application must actually write its logs to `/var/log/cb.log`

Step 4 — Check how the application currently writes logs

Your Dockerfile currently has:

```
ENTRYPOINT ["/hello_world"]
```

The application probably writes logs to **stdout/stderr**.

Check:

```
docker logs cricbuzz-hello-world
```

If you see application logs there, the simplest solution is to redirect the container's `stdout/stderr` to the host file.

Step 5 — Configure Docker Compose to write `stdout/stderr` to `/var/log/cb.log`

For your assignment, I recommend using a small wrapper script.

Create:

```
vi start.sh
```

```
ubuntu@ip-172-31-28-253: ~/cricbuzz-docker-hello-world
#1/bin/sh
/hello_world >> /var/log/cb.log 2>&1
~
~
~
~
~
~
~
```

Step 6 — Modify the Dockerfile

Open:

vi Dockerfile

```
ubuntu@ip-172-31-28-253: ~/cricbuzz-docker-hello-world
FROM ghcr.io/vincetse/scratch
LABEL maintainer "Vince Tse <vincetse@users.noreply.github.com>"
COPY ./hello_world .
COPY ./start.sh .

ENV GIN_MODE release

EXPOSE 8080

#ENTRYPOINT ["/hello_world"]
ENTRYPOINT ["/start.sh"]
~
~
~
~
~
```

Step 7 — Easier solution for your assignment

If your requirement simply means:

"The Docker application logs should be saved in /var/log/cb.log on the EC2 server."

then we can use Docker's logging mechanism without changing the application.

First remove the previous container:

docker compose down

Then create the log file:

sudo touch /var/log/cb.log

Now configure Docker's logging driver to write its logs to the Docker JSON log, but that **does not directly create /var/log/cb.log**.

So if the assignment specifically requires this exact file:

`/var/log/cb.log`

the most reliable method is to redirect the application's output from the host.

Step 8 — Use Docker's logs command

You can test whether the application produces logs:

```
docker logs cricbuzz-hello-world
```

Then redirect them:

```
docker logs cricbuzz-hello-world > /var/log/cb.log 2>&1
```

Because `/var/log` requires root:

```
docker logs cricbuzz-hello-world | sudo tee /var/log/cb.log > /dev/null
```

Check:

```
sudo cat /var/log/cb.log
```

This gives you:

`/var/log/cb.log`

containing the current Docker logs.

But note

This is a **snapshot**, not continuous logging.

If the application generates new logs later, `/var/log/cb.log` will not automatically update.

Step 9 — Continuous logging

For continuous logging, run:

```
ubuntu@ip-172-31-28-253: ~  
ubuntu@ip-172-31-28-253:~$ docker logs -f cricbuzz-hello-world | sudo tee -a /var/log/cb.log  
[GIN] 2026/09/16 - 09:21:17 | 200 | 47.098µs | 223.228.99.204 | GET | "/  
[GIN] 2026/09/16 - 09:21:18 | 404 | 23.271µs | 223.228.99.204 | GET | "/favicon.ico"
```

6. Write the Log rotate configuration for the above file

Now that your log file is:

/var/log/cb.log

the next step is to configure **Linux Logrotate** so the file does not grow indefinitely.

We will configure:

- Rotate the log **daily**
- Keep **7 old log files**
- Compress old logs
- Create a new cb.log
- Use copytruncate so the application can continue writing without restarting
- Ignore the file if it doesn't exist

The result will look like:

/var/log/cb.log

/var/log/cb.log.1

/var/log/cb.log.2.gz

/var/log/cb.log.3.gz

...

Step 1 — Check whether Logrotate is installed

Run:

logrotate --version

If you see a version such as:

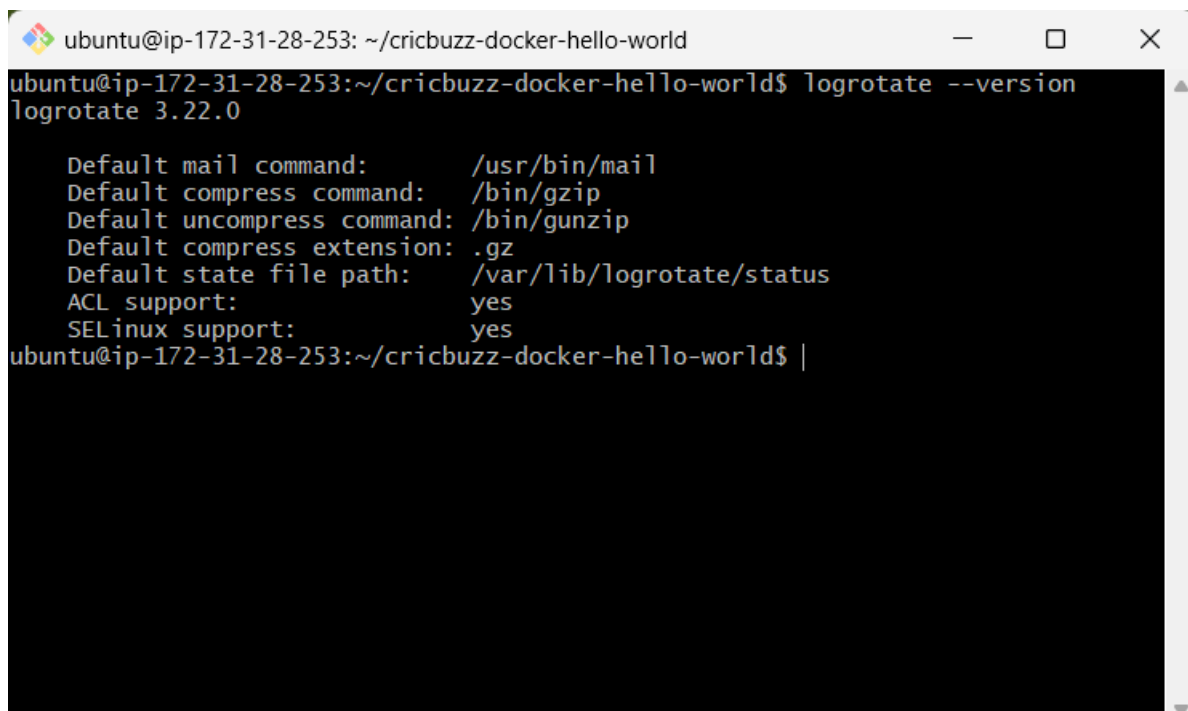
logrotate 3.x.x

it is installed.

If you get command not found, install it:

sudo apt update

sudo apt install logrotate -y

A terminal window with a title bar showing 'ubuntu@ip-172-31-28-253: ~/cricbuzz-docker-hello-world'. The terminal output shows the command 'logrotate --version' being executed, resulting in 'logrotate 3.22.0'. Below this, the default configuration for logrotate is displayed, including the mail command, compress/uncompress commands, extension, state file path, and support for ACL and SELinux. The prompt returns to the shell.

```
ubuntu@ip-172-31-28-253: ~/cricbuzz-docker-hello-world
ubuntu@ip-172-31-28-253:~/cricbuzz-docker-hello-world$ logrotate --version
logrotate 3.22.0

Default mail command:      /usr/bin/mail
Default compress command:  /bin/gzip
Default uncompress command: /bin/gunzip
Default compress extension: .gz
Default state file path:   /var/lib/logrotate/status
ACL support:               yes
SELinux support:           yes
ubuntu@ip-172-31-28-253:~/cricbuzz-docker-hello-world$ |
```

Step 2 — Check your log file

Run:

sudo ls -lh /var/log/cb.log

```
ubuntu@ip-172-31-28-253: ~/cricbuzz-docker-hello-world
ubuntu@ip-172-31-28-253:~/cricbuzz-docker-hello-world$ sudo ls -lh /var/log/cb.log
-rw-r--r-- 1 root root 179 Sep 16 11:43 /var/log/cb.log
ubuntu@ip-172-31-28-253:~/cricbuzz-docker-hello-world$
```

Step 3 — Create the Logrotate configuration

Create a configuration specifically for Cricbuzz:

```
sudo vi /etc/logrotate.d/cricbuzz
```

```
/var/log/cb.log {
```

```
    daily
```

```
    rotate 7
```

```
    compress
```

```
    delaycompress
```

```
    missingok
```

```
    notifempty
```

```
    copytruncate
```

```
    create 0644 root root
```

```
}
```

Save:

Esc

:wq

Enter

Step 4 — Understand the configuration

Log file

/var/log/cb.log

This tells Logrotate which file to manage.

Rotate daily

daily

Logrotate checks this file every day.

Keep 7 rotations

rotate 7

It keeps seven old versions.

For example:

cb.log

cb.log.1

cb.log.2.gz

cb.log.3.gz

cb.log.4.gz

cb.log.5.gz

cb.log.6.gz

cb.log.7.gz

Compress old logs

compress

Old logs are compressed using gzip.

For example:

cb.log.2.gz

Don't compress the newest rotated log immediately

delaycompress

This keeps the most recently rotated log uncompressed until the next rotation.

Don't report an error if the file doesn't exist

missingok

Don't rotate an empty file

notifempty

Continue writing without restarting the application

copytruncate

This is particularly useful when the process keeps the log file open. Logrotate copies the existing contents and then truncates the original file.

Permissions for the new file

create 0644 root root

A newly created log file will have:

-rw-r--r--

and ownership:

root root

Step 5 — Verify the configuration

Run:

sudo logrotate -d /etc/logrotate.d/cricbuzz

-d means **debug/dry run**. It does not actually rotate the file.

Look for something similar to:

considering log /var/log/cb.log

and:

log does not need rotating

or information about what would happen during rotation.

There should be **no configuration error**.

```
ubuntu@ip-172-31-28-253: ~/cricbuzz-docker-hello-world
ubuntu@ip-172-31-28-253:~/cricbuzz-docker-hello-world$ sudo logrotate -d /etc/logro
grotate.d/cricbuzz
warning: logrotate in debug mode does nothing except printing debug messages! C
onsider using verbose mode (-v) instead if this is not what you want.

reading config file /etc/logrotate.d/cricbuzz
Reading state from file: /var/lib/logrotate/status
Allocating hash table for state file, size 64 entries
Creating new state
Creating new state
Creating new state
Creating new state
Creating new state
Creating new state
Creating new state
Creating new state
Creating new state
Creating new state
Creating new state
Creating new state
Creating new state
Creating new state
Creating new state
Creating new state
Creating new state
Creating new state
Creating new state
Creating new state
Creating new state
Creating new state
Creating new state
Handling 1 logs

rotating pattern: /var/log/cb/log after 1 days empty log files are not rotated,
(7 rotations), old logs are removed
considering log /var/log/cb/log
Creating new state
ubuntu@ip-172-31-28-253:~/cricbuzz-docker-hello-world$
```

Step 6 — Force a rotation for testing

For your lab, we can test it immediately instead of waiting until tomorrow.

Run:

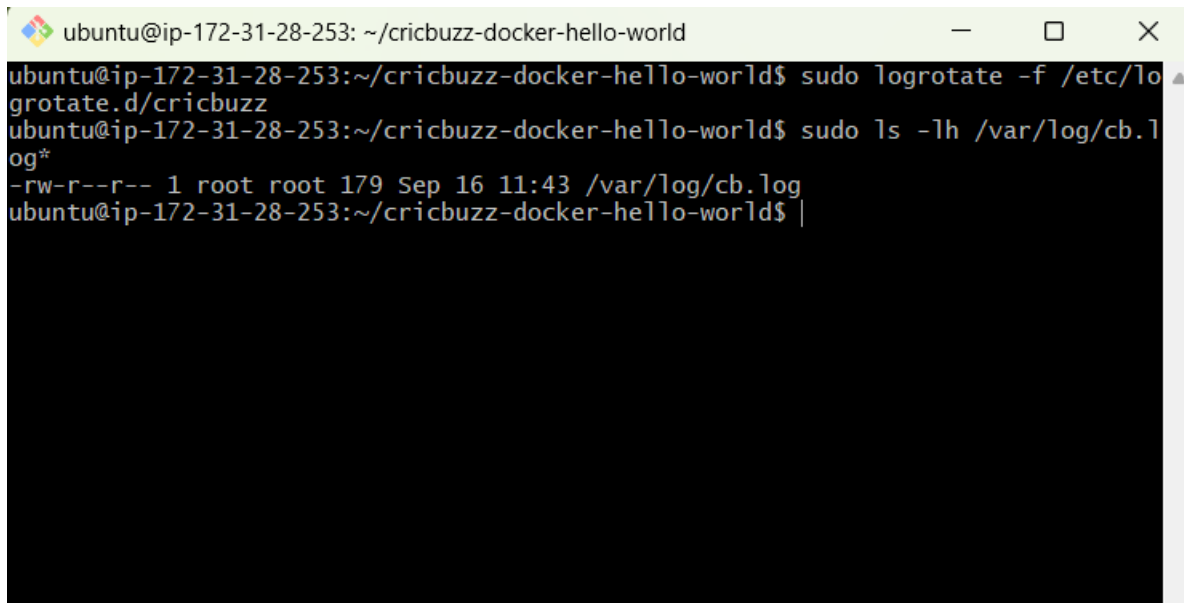
```
sudo logrotate -f /etc/logrotate.d/cricbuzz
```

The -f means **force rotation**.

Step 7 – Check the result

Run:

```
sudo ls -lh /var/log/cb.log*
```

A terminal window with a green title bar showing the command 'sudo logrotate -f /etc/logrotate.d/cricbuzz' and its output. Below that, the command 'sudo ls -lh /var/log/cb.log*' is entered, showing the output for the file /var/log/cb.log with permissions -rw-r--r-- and size 179 bytes.

```
ubuntu@ip-172-31-28-253: ~/cricbuzz-docker-hello-world
ubuntu@ip-172-31-28-253:~/cricbuzz-docker-hello-world$ sudo logrotate -f /etc/logrotate.d/cricbuzz
ubuntu@ip-172-31-28-253:~/cricbuzz-docker-hello-world$ sudo ls -lh /var/log/cb.log*
-rw-r--r-- 1 root root 179 Sep 16 11:43 /var/log/cb.log
ubuntu@ip-172-31-28-253:~/cricbuzz-docker-hello-world$ |
```

Step 8 — Check the contents

Current log:

```
sudo cat /var/log/cb.log
```

Rotated log:

```
sudo cat /var/log/cb.log.1
```

Compressed log:

```
sudo zcat /var/log/cb.log.2.gz
```

Step 9 — Check Logrotate's global configuration

Ubuntu normally runs Logrotate automatically through its scheduled mechanism.

Check:

```
systemctl status logrotate.timer --no-pager
```

```
ubuntu@ip-172-31-28-253: ~/cricbuzz-docker-hello-world
ubuntu@ip-172-31-28-253:~/cricbuzz-docker-hello-world$ sudo cat /var/log/cb.log
[GIN] 2026/09/16 - 09:21:17 | 200 | 47.098µs | 223.228.99.204 | GET "/"
[GIN] 2026/09/16 - 09:21:18 | 404 | 23.271µs | 223.228.99.204 | GET "/favicon.ico"
ubuntu@ip-172-31-28-253:~/cricbuzz-docker-hello-world$ systemctl status logrotate.timer --no-pager
● logrotate.timer - Daily rotation of log files
   Loaded: loaded (/usr/lib/systemd/system/logrotate.timer; enabled; preset: enabled)
   Active: active (waiting) since Wed 2026-09-16 05:46:17 UTC; 23h ago
     Invocation: a223a07696294f21bfe931f65b57ae74
       Trigger: Fri 2026-09-18 00:18:26 UTC; 19h left
     Triggers: ● logrotate.service
        Docs: man:logrotate(8)
              man:logrotate.conf(5)

Sep 16 05:46:17 ip-172-31-28-253 systemd[1]: Started logrotate.timer - Daily rotation of log files.
Hint: Some lines were ellipsized, use -l to show in full.
ubuntu@ip-172-31-28-253:~/cricbuzz-docker-hello-world$ systemctl list-timers | grep logrotate
Fri 2026-09-18 00:18:26 UTC              19h Thu 2026-09-17 00:04:06 UTC 4h 53min
ago logrotate.timer                  logrotate.service
ubuntu@ip-172-31-28-253:~/cricbuzz-docker-hello-world$
```

Step 10 — Final configuration

Your file:

/etc/logrotate.d/cricbuzz

```
ubuntu@ip-172-31-28-253:~/cricbuzz-docker-hello-world$ cat /etc/logrotate.d/cricbuzz
/var/log/cb/log {
    daily
    rotate 7
    compress
    delaycompress
    missingok
    notifempty
    copytruncate
    create 0644 root root
}
ubuntu@ip-172-31-28-253:~/cricbuzz-docker-hello-world$
```

